Case Study on the Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Roshan S

241501170

AIML

Abstract

This case study analyses where low-code platforms such as Zoho Creator fall short when applied to a complex, large-scale enterprise use case. A fictional gym chain tries to extend a successful single-gym membership application into a group-wide central membership, access-control and pricing system. The study identifies the limitations that appear in performance, concurrency, algorithms, integration, data volume, portability and governance, and recommends a hybrid architecture as the practical solution.

1. Introduction

Low-code platforms are very effective for workflow-centred business applications, and success with a small application often encourages organizations to use the same platform for much larger systems. This is where limitations appear. A case study helps show which requirements push a system beyond what a low-code platform handles comfortably, and how to respond.

2. Case Scenario

The organization in this scenario is fictional and the case is created for academic study. FitNation Gyms is a gym chain with 150 branches in several countries. After the Gym Membership Enrollment & Front Desk Management application built in Zoho Creator, with its nine reports, worked well at one gym, management proposes to extend it into a group-wide central membership and access-control system with these requirements:

- Real-time membership status and class-slot availability synchronized across all branches, the mobile app and door-access turnstiles.
- Thousands of concurrent check-ins and class bookings at peak evening hours with no overbooked classes.

- Dynamic pricing and churn prediction driven by machine learning models.
- Integration with legacy billing, payment gateway, payroll and accounting systems.
- Strict compliance with data protection, payment security and audit rules in each country.

3. Analysis of Limitations

The requirements are examined against the typical limits of a low-code platform.

Area	Limitation on a Low-Code Platform	Impact on the Use Case
Performance and scalability	Scripts run under execution-time and API-call limits; fine-grained query tuning and caching are not available	Peak-hour check-in bursts and large membership recalculations may be slow or fail
Concurrency and transactions	Limited control over transactions and locking	Risk of overbooked classes and duplicate check-ins under simultaneous requests
Complex algorithms	Scripting suits business rules, not heavy computation or ML inference	Dynamic pricing and churn prediction must be built outside the platform
Integration depth	Connectors and REST cover common cases; legacy protocols, turnstile hardware and event streaming are hard	Custom middleware becomes necessary
Data volume	Storage and reporting are tuned for business-scale data	Slow reports on years of check-in history; external data warehouse needed
UI customization	Limited to the components the builder offers	Brand-specific member app journeys are hard to achieve
Vendor lock-in	Logic, data model and workflows depend on the vendor runtime	Costly migration if pricing or features change

Area	Limitation on a Low-Code Platform	Impact on the Use Case
Governance at scale	Many makers and apps can cause sprawl and inconsistent security	Compliance and audit risk across countries
Testing and DevOps	Limited automated testing, version control and CI/CD	Harder to enforce quality gates in a large team

The most serious risks are concurrency and performance. A class-booking and access system must never admit more members than a class allows or double-process a check-in, and this needs close control over transactions and locking that a low-code runtime does not fully expose. Dynamic pricing and churn prediction are computation-heavy and fall outside what a business-rule scripting language is designed to do.

4. Mitigation Strategies

- Adopt a hybrid architecture: keep forms, workflows, approvals and dashboards in low-code, and move high-load or algorithmic parts to pro-code services.
- Use an integration layer such as an API gateway between the low-code application and legacy systems and turnstile hardware.
- Move large-scale analytics to a data warehouse.
- Establish governance with role-based access, separate development, staging and production environments, naming standards and central review.
- Run a proof of concept and load tests before committing the whole enterprise to a platform.
- Plan an exit strategy by keeping data exportable and business logic documented.

5. Recommended Decision

A full migration of the central membership and access engine to a purely low-code platform is not recommended. The low-code application should be kept for front-desk workflows, enrollment forms and reporting at each branch, while the real-time class-slot and access engine and the pricing model are built as pro-code services connected through APIs.

Component	Recommended Approach
Enrollment forms, front-desk reports, the nine-report suite, approvals	Low-code (Zoho Creator)
Real-time class booking and turnstile access engine	Pro-code service exposed through an API
Dynamic pricing and churn prediction models	Pro-code or ML service (for example on Zoho Catalyst)
Group-wide analytics	External data warehouse and BI tool
Legacy billing and payment connectivity	Integration layer or API gateway

6. Conclusion

Low-code platforms are excellent for workflow-centred applications but are not a universal solution. Highly concurrent, algorithm-heavy and deeply integrated enterprise systems expose limits in performance, control and portability. Recognizing these limits early and adopting a hybrid architecture lets an organization keep the speed of low-code without giving up reliability.

7. Summary

The case of FitNation Gyms shows that a low-code application that succeeds at small scale cannot simply be stretched to an enterprise membership and access system. Limits in performance, concurrency, algorithms, integration depth, data volume, customization, lock-in, governance and DevOps all apply. A hybrid design, supported by governance and load testing, gives the best balance.

8. Learning Outcome

This case study develops the ability to judge whether a low-code platform is suitable for a given enterprise requirement, to recognize its technical and organizational limits, and to propose a hybrid architecture that combines low-code with pro-code extensions.